Alpaca Algorithmic Trading Guide

https://gemini.google.com/app/ffb4d4b9366d2ee5

User prompt: Tech Trend (QQQBreakout),stock_momentum

Response: Save this unified script as **`multi_trader.py`**.

This script isolates **QQQBreakout** (45% capital allocation) and **StockMomentum** (45% capital allocation) onto a single Alpaca paper account, reserving a 10% cash buffer. Crucially, it replaces blanket liquidations with **targeted exits** (`self.sell_all(symbol)`), ensuring neither strategy closes the other's positions.

```
import os
import sys
from dotenv import load_dotenv

# Prevent OpenBLAS memory allocation overhead on Windows
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

load_dotenv()

from lumibot.brokers import Alpaca
from lumibot.strategies import Strategy
from lumibot.traders import Trader


# ========================================
# STRATEGY 1: Tech Trend (QQQ Breakout)
# ========================================
class QQQBreakoutLive(Strategy):
    parameters = {
        "symbol": "QQQ",
        "sma_period": 20,
        "allocation": 0.45,  # 45% Portfolio Cap
        "buffer_pct": 0.005,  # 0.5% Whipsaw Buffer
    }

    def initialize(self):
        self.sleeptime = "1M"

    def on_trading_iteration(self):
        symbol = self.parameters["symbol"]
        sma_period = self.parameters["sma_period"]

        price = self.get_last_price(symbol)
        bars = self.get_historical_prices(symbol, sma_period, "day")

        if bars is None or len(bars.df) < sma_period or price is None:
            return

        sma = float(bars.df["close"].mean())
        position = self.get_position(symbol)

        # Entry: QQQ crosses above 20-day SMA
        if position is None and price > sma:
```

```
            target_cash = self.get_portfolio_value() * self.parameters["allocation"]
            qty = int(target_cash // price)
            if qty > 0:
                self.submit_order(self.create_order(symbol, qty, "buy"))
                self.log_message(f"[QQQ-Breakout] BUY {qty} {symbol} @ ${price:.2f} | 20 SMA: ${sma:.2f}")

        # Targeted Exit: Liquidates ONLY QQQ if price falls 0.5% below SMA
        elif position is not None and price < (sma * (1 - self.parameters["buffer_pct"])):
            self.log_message(f"[QQQ-Breakout] EXIT {symbol} @ ${price:.2f} (0.5% below SMA)")
            self.sell_all(symbol)


# ========================================
# STRATEGY 2: Multi-Stock Relative Momentum
# ========================================
class BasketMomentumLive(Strategy):
    parameters = {
        # High-liquidity tech leaders distinct from the core index
        "universe": ["NVDA", "AAPL", "MSFT"],
        "lookback_days": 10,
        "allocation": 0.45,  # 45% Total Cap (15% per stock max)
    }

    def initialize(self):
        self.sleeptime = "5M"

    def on_trading_iteration(self):
        universe = self.parameters["universe"]
        lookback = self.parameters["lookback_days"]
        slot_budget = (self.get_portfolio_value() * self.parameters["allocation"]) / len(universe)

        for symbol in universe:
            bars = self.get_historical_prices(symbol, lookback + 1, "day")
            if bars is None or len(bars.df) < (lookback + 1):
                continue

            current_price = self.get_last_price(symbol)
            if not current_price:
                continue

            # Calculate 10-day Rate of Change (ROC)
            past_price = float(bars.df["close"].iloc[0])
            momentum = (current_price - past_price) / past_price

            position = self.get_position(symbol)

            # Buy on positive momentum if position is empty
            if position is None and momentum > 0.02:  # +2% gain over 10 days
                qty = int(slot_budget // current_price)
                if qty > 0:
                    self.submit_order(self.create_order(symbol, qty, "buy"))
                    self.log_message(f"[Momentum] BUY {qty} {symbol} | 10D Momentum: {momentum*100:.2f}%")

            # Targeted Exit: Sell ONLY this specific symbol if momentum turns negative
            elif position is not None and momentum < 0.0:
```

```
                self.log_message(f"[Momentum] EXIT {symbol} | Momentum dropped below 0%")
                self.sell_all(symbol)


# ========================================
# UNIFIED LIVE RUNNER
# ========================================
if __name__ == "__main__":
    api_key = os.getenv("ALPACA_API_KEY")
    api_secret = os.getenv("ALPACA_API_SECRET")
    is_paper = os.getenv("ALPACA_IS_PAPER", "true").lower() == "true"

    if not api_key or not api_secret:
        print("Missing Alpaca API credentials in .env")
        sys.exit(1)

    alpaca_config = {
        "API_KEY": api_key,
        "API_SECRET": api_secret,
        "PAPER": is_paper,
    }

    broker = Alpaca(alpaca_config)

    # Instantiate both strategies with the shared broker
    qqq_strategy = QQQBreakoutLive(broker=broker)
    momentum_strategy = BasketMomentumLive(broker=broker)

    # Orchestrate both concurrently in the same runtime
    trader = Trader()
    trader.add_strategy(qqq_strategy)
    trader.add_strategy(momentum_strategy)

    print("=" * 60)
    print(" Active Multi-Trader: [QQQ Breakout] + [Basket Momentum]")
    print(" Account: Alpaca Paper | Allocation: 45% / 45% / 10% Cash")
    print("=" * 60)

    trader.run_all()
```

Key Protections in This Implementation

- **45% / 45% / 10% Partitioning:** The portfolio value is dynamically checked before orders are calculated, ensuring neither bot exhausts the account's cash or triggers margin rejections.
- **Isolated Exits:** `self.sell_all(symbol)` targets only the specific ticker owned by that strategy. If `NVDA` turns weak and sells, your `QQQ` position remains completely untouched.
- **Staggered Polling:** `QQQBreakout` polls every 1 minute (`1M`), while `BasketMomentum` polls every 5 minutes (`5M`), keeping total REST calls comfortably below Alpaca's 200 requests/minute limit.

Would you like to start this script now to confirm that both strategies register on the broker and enter sleep mode until Monday's opening bell?

User prompt: where is SPY?

Response: In the previous script, **SPY** was swapped out for individual high-momentum stocks (`NVDA`, `AAPL`, `MSFT`) to avoid overlapping broad-market exposure with QQQ.

To trade **SPY** directly with the momentum strategy alongside QQQ, change the universe in `BasketMomentumLive` to `["SPY"]`:

```
# ========================================
# STRATEGY 2: SPY Momentum
# ========================================
class BasketMomentumLive(Strategy):
    parameters = {
        "universe": ["SPY"],  # Trades SPY directly alongside QQQ
        "lookback_days": 10,
        "allocation": 0.45,  # 45% of total portfolio cash
    }
```

How Both Strategies Run Side-by-Side

Strategy	Target Symbol	Allocation Cap	Entry Trigger	Exit Trigger
QQQBreakoutLive	QQQ	45%	Price > 20-day SMA	Price < (SMA × 0.995)
MomentumLive	SPY	45%	10-day ROC > +2%	10-day ROC < 0%
Cash Reserve	USD Cash	10%	Unallocated buffer	Prevents broker margin errors

Updated Strategy 2 Block in `multi_trader.py`

Update `BasketMomentumLive` in your file:

```
class BasketMomentumLive(Strategy):
    parameters = {
        "universe": ["SPY"],
        "lookback_days": 10,
        "allocation": 0.45,
    }

    def initialize(self):
        self.sleeptime = "5M"

    def on_trading_iteration(self):
        universe = self.parameters["universe"]
        lookback = self.parameters["lookback_days"]
        slot_budget = (self.get_portfolio_value() * self.parameters["allocation"]) / len(universe)

        for symbol in universe:
            bars = self.get_historical_prices(symbol, lookback + 1, "day")
            if bars is None or len(bars.df) < (lookback + 1):
                continue

            current_price = self.get_last_price(symbol)
            if not current_price:
                continue

            past_price = float(bars.df["close"].iloc[0])
            momentum = (current_price - past_price) / past_price

            position = self.get_position(symbol)
```

```
            # Buy SPY if 10-day return > 2%
            if position is None and momentum > 0.02:
                qty = int(slot_budget // current_price)
                if qty > 0:
                    self.submit_order(self.create_order(symbol, qty, "buy"))
                    self.log_message(f"[SPY-Momentum] BUY {qty} {symbol} | 10D Return: {momentum*100:.2f}%")

            # Exit SPY if 10-day momentum turns negative
            elif position is not None and momentum < 0.0:
                self.log_message(f"[SPY-Momentum] EXIT {symbol} | Momentum fell below 0%")
                self.sell_all(symbol)
```

User prompt: what are all this files then, does the bot uses them?

Response: Some of these files are the core engine your bot uses on every run, while others are optional data connectors, sample strategies, or developer tests.

1. `entities/` — The Core Engine (Used on Every Run)

Yes, your bot actively uses these files.

These files define the fundamental Python objects and data structures of LumiBot. Whenever your strategy checks a price or places a trade, it relies on this folder:

- **`bars.py` & `bar.py`:** Manages historical candlestick data and dataframes (this is where the `self.df["close"].iloc[-1]` accessor runs).
- **`order.py` & `position.py`:** Defines the order objects created when calling `self.create_order()` and tracks active share counts when calling `self.get_position()`.
- **`asset.py` & `quote.py`:** Standardizes how tickers, asset classes (stocks, crypto, options), and live bid/ask quotes are represented.
- **`trading_fee.py` & `trading_slippage.py`:** Simulates real-world friction like commission fees and spread slippage.

2. `backtesting/` — Data Feeds & Simulators (Used On-Demand)

Your bot uses only the specific data source you import.

These files are adapters connecting LumiBot's backtesting engine to different historical data providers:

- **`alpaca_backtesting.py`:** The exact file executed when you ran `QQQBreakout.backtest(AlpacaBacktesting, ...)`.
- **`backtesting_broker.py`:** The simulated broker that fills virtual orders, tracks cash balances, and calculates portfolio metrics during a backtest.
- **`yahoo_backtesting.py`, `polygon_backtesting.py`, `databento_*.py`, `thetadata_*.py`:** Alternative data providers. If you don't import them, they remain completely idle.

3. `example_strategies/` — Templates & Demos (Not Used Unless Run)

No, your live bot does not use these files unless you explicitly run or import them.

This is a library of pre-built trading templates created by the Lumiwealth team for inspiration and reference:

- **AI Agent Bots (`ai_trading_team_*.py`):** Multi-agent frameworks modeling investment styles (Ray Dalio macro risk-parity, Citadel multi-manager pods, Warren Buffett value investing).
- **Portfolio Allocators (`classic_60_40.py`, `crypto_50_50.py`):** Fixed-allocation rebalancing engines.
- **Derivatives & Contracts (`polymarket_prediction_contract.py`, `options_hold_to_expiry.py`):** Specialized logic for options and event contracts.

- **Execution Utilities (`stock_bracket.py`, `schedule_function.py`):** Examples demonstrating stop-loss/take-profit brackets and cron-style timed functions.

4. `tests/` — Developer Test Suite (Never Used at Runtime)

No, these are never used during live trading or standard backtesting.

This directory contains test suites (`pytest`) used by LumiBot's software engineers to verify that updates to the library don't introduce bugs:

- **`test_*.py`:** Automated tests checking order execution, margin accounting, and indicator precision.
- **`baselines.json` & `history.csv`:** Benchmark reference numbers to detect performance regressions across software updates.

Summary Table

Folder	What It Is	Does Your Bot Use It?
`entities/`	Core internal data models (orders, positions, bars)	**Yes** — runs constantly in the background.
`backtesting/`	Data adapters and virtual exchange engine	**Partially** — only the adapter you import (`AlpacaBacktesting`).
`example_strategies/`	Standalone sample strategy scripts	**No** — reference code and templates only.
`tests/`	Developer CI/CD unit and regression tests	**No** — test scripts for library developers.

Would you like to explore any of the strategies from `example_strategies` (such as the bracket order system or the 60/40 rebalancer) to see how its logic is structured?

User prompt: (venv) PS D:\alpaca\my_algo_bot> python live_trader.py 2026-09-12 21:03:49,564 | INFO | LumiBot v4.5.91 starting 2026-09-12 21:03:49,614 | INFO | .env file loaded from: D:\alpaca\my_algo_bot\.env 2026-09-12 21:03:51,784 | INFO | [unknown] Waiting for the socket stream connection to be established, method _stream_established must be called 2026-09-12 21:03:51,921 | INFO | [unknown] Waiting for the socket stream connection to be established, method _stream_established must be called 2026-09-12 21:03:53,896 | INFO | [unknown] LUMIBOT_TELEMETRY {"broker":"alpaca","broker_snapshot": {"orders_canceled":0,"orders_error":0,"orders_filled":0,"orders_new":0,"orders_partial":0,"orders_placeholder":0,"orders_unprocessed":0,"positions_tracked":0,"trade_events_rows":0},"strategy_name":a 2026-09-12 21:03:53,934 | INFO | [unknown] LUMIBOT_TELEMETRY {"broker":"alpaca","broker_snapshot": {"orders_canceled":0,"orders_error":0,"orders_filled":0,"orders_new":0,"orders_partial":0,"orders_placeholder":0,"orders_unprocessed":0,"positions_tracked":0,"trade_events_rows":0},"strategy_name":a == Starting QQQ Live Bot in PAPER TRADING mode Polling Interval: 1 Minute | Target: QQQ == 2026-09-12 21:03:54,775 | INFO | [QQQBreakoutLive] Strategy QQQBreakoutLive is initializing 2026-09-12 21:03:56,642 | WARNING | lumibot_logger.py:warning:613 | [QQQBreakoutLive] LUMIWEALTH_API_KEY not set. Not sending an update to the cloud because lumiwealth_api_key is not set. If you would like to be able to track your bot performance on www.botspot.trade, please set the lumiwealth_api_key parameter in the strategy initialization or the LUMIWEALTH_API_KEY environment variable. 2026-09-12 21:03:56,920 | INFO | [QQQBreakoutLive] Sleeping until the market opens

Response: Your setup initialized cleanly: your Alpaca paper API credentials connected, the WebSocket event streams were verified, and the bot is now hibernating until the opening bell on Monday.

Log Breakdown

- **`Waiting for the socket stream connection...` / `LUMIBOT_TELEMETRY`:** The broker handshake succeeded. LumiBot opened a persistent bi-directional WebSocket connection to Alpaca to listen for real-time order fills, position updates, and cancellations.
- **`LUMIWEALTH_API_KEY not set` (Warning):** This is non-fatal and safe to ignore. It only applies if you use Lumiwealth's paid commercial cloud tracking dashboard (`botspot.trade`).
- **`Sleeping until the market opens`:** Because US stock exchanges (NASDAQ/NYSE) are closed on weekends, LumiBot automatically checks the exchange calendar and halts polling until the regular session begins.

What Happens Next

- **Market Open Timing:** During Daylight Saving Time, US equity regular trading hours (9:30 AM to 4:00 PM EDT) run from **7:00 PM to 1:30 AM IST**.

- **Automatic Wake-Up:** At **7:00 PM IST on Monday**, the script will wake up on its own, download the last 20 daily closing bars for QQQ, compute the 20-day SMA, and begin polling live quotes every 60 seconds.
- **Order Execution:** If QQQ's price trades above the 20-day SMA, the bot will immediately place a market buy order for 90% of your paper portfolio value, visible in real time on your Alpaca Paper Dashboard.

Your Options Now

1. **Leave the terminal open:** If your machine remains awake and connected to the internet, the bot will run automatically on Monday evening without any intervention.
2. **Close it for now:** Press `Ctrl + C` in PowerShell to safely terminate the process. You can start it again anytime before 7:00 PM IST Monday by running `python live_trader.py`.

Would you like to keep the bot in sleep mode until Monday, or would you prefer to run a quick 24/7 crypto test script to see a real order fill on your Alpaca paper dashboard right now?

User prompt: how does the bot takes trades based on which stratagie?

Response: LumiBot evaluates each strategy as an independent, isolated thread within the `Trader` orchestrator. Each strategy runs its own decision loop, reads its own market data, and issues orders independently to your Alpaca account.

The Decision Loop Inside Each Strategy

Every strategy subclassing `Strategy` follows a 4-step execution cycle:

```
[Timer / sleeptime triggers]
       ↓
[Fetch Data: get_last_price() / get_historical_prices()]
       ↓
[Evaluate Technical Signal: SMA, ROC, RSI, etc.]
       ↓
[Check Existing Holdings: get_position(symbol)]
       ↓
[Calculate Quantity & Submit Order: submit_order()]
```

1. **Independent Clocks (`sleeptime`):** Each strategy runs on its own schedule. `QQQBreakoutLive` sets `self.sleeptime = "1M"` (evaluating every 60 seconds), while `BasketMomentumLive` sets `self.sleeptime = "5M"` (evaluating every 5 minutes). The `Trader` manages these threads separately without blocking each other.
2. **Strategy-Specific Data Queries:** When `on_trading_iteration()` fires, the strategy requests market data for only its configured parameter (`symbol = self.parameters["symbol"]`).
3. **Logic Gating:** The strategy compares the live price against mathematical rules:
   - **QQQ Breakout:** Triggers a buy only if `current_price > 20_day_SMA` and `get_position("QQQ") is None`.
   - **SPY Momentum:** Triggers a buy only if `10_day_ROC > 2%` and `get_position("SPY") is None`.
4. **Order Execution:** If a condition is met, the strategy calls `self.create_order()` and `self.submit_order()`. The order is dispatched over Alpaca's REST API directly to your paper account.

How LumiBot Distinguishes and Isolates Trades

Mechanism	How It Works in Code	What Prevents Collisions
Strategy Instance	Each strategy is an instantiated class (`strat_qqq = QQQBreakoutLive(...)`).	Internal parameters, states, and indicators remain scoped to that specific class instance.
Position Scoping	`self.get_position("QQQ")` checks only the target ticker.	QQQ never checks or reads SPY positions; SPY never checks QQQ.

Mechanism	How It Works in Code	What Prevents Collisions
Capital Allocation	`self.get_portfolio_value() * self.parameters["allocation"]`	Dynamic calculation ensures each strategy only sizes orders using its allowed 45% slice of account equity.
Targeted Liquidation	`self.sell_all("QQQ")` specifies the exact asset to close.	Passing the symbol prevents the bot from liquidating positions opened by the other strategy.
Broker Order Tagging	LumiBot tags outgoing orders with internal strategy identifiers and client order IDs.	The broker can match execution events and WebSocket trade updates back to the strategy that submitted them.

Execution Scenario

- **9:31 AM EST:** `QQQBreakoutLive` runs its 1-minute check. QQQ crosses above its 20-day SMA ($480.00 > $478.50). The strategy calculates 45% of available cash, buys 93 shares of QQQ, and logs the execution.
- **9:35 AM EST:** `BasketMomentumLive` runs its 5-minute check. SPY's 10-day rate of change is +2.4%. It checks its 45% allocation, buys 75 shares of SPY, and submits the order.
- **10:15 AM EST:** QQQ dips sharply, crossing 0.5% below its 20-day SMA. `QQQBreakoutLive` executes `self.sell_all("QQQ")`. Only QQQ is sold to cash; your SPY momentum trade remains active and untouched.

Would you like to add a customized logging wrapper or Discord/Telegram webhook to receive immediate push notifications specifying which strategy executed each order?

User prompt: so bot checks all the stratagies and place order when one stratagies passes?

Response: Each strategy acts as an independent worker running its own loop. When you attach multiple strategies to `Trader`, they run in parallel rather than waiting for each other or requiring mutual agreement.

Whenever any single strategy's technical criteria evaluate to true, that strategy immediately submits an order for its assigned symbol to Alpaca.

How the Multi-Strategy Workflow Operates

- **Independent Timers:** Each strategy checks market data on its own clock (`sleeptime`). One can check every 60 seconds while another checks every 5 minutes.
- **Instant Action on Trigger:** If Strategy A's breakout condition passes at 9:31 AM, Strategy A immediately fires a buy order for QQQ. It does not wait to see what Strategy B is doing.
- **Zero Cross-Interference:** If Strategy B fails its momentum check at 9:35 AM, it does nothing and goes back to sleep. Strategy A's open QQQ position remains active and completely unaffected.

Timeline Example During Live Trading

Time	Strategy 1: QQQ Breakout (`1M` loop)	Strategy 2: SPY Momentum (`5M` loop)	Resulting Portfolio Action
9:31 AM	**Signal PASSES** (QQQ > 20 SMA)	*Sleeping*	Buys 45% allocation of **QQQ**.
9:35 AM	Evaluates: Already long QQQ (No action)	**Signal FAILS** (SPY ROC < 2%)	No trade executed. Holds QQQ; cash remains idle.
9:40 AM	Evaluates: Holding QQQ (No action)	**Signal PASSES** (SPY ROC > 2%)	Buys 45% allocation of **SPY**.
10:15 AM	**Exit PASSES** (QQQ drops 0.5% below SMA)	Evaluates: Holding SPY (No action)	**Sells QQQ only.** SPY position stays 100% intact.

The One Critical Rule

Because any strategy can fire an order the second it passes, each strategy must have a hard **capital cap** (such as 45% each). If a strategy were allowed to use 100% of the account, whichever strategy triggered first would lock up all buying power, causing Alpaca to reject the second strategy's order when its setup finally passes.

Would you like to run `multi_trader.py` with both QQQ and SPY active so both loops are primed for Monday's market open?

User prompt: yes and does it also places orders on long and short ETFs ?

Response: Yes. The bot can trade both standard long ETFs and inverse (short) ETFs.

There are two primary ways to trade the downside on Alpaca using LumiBot:

1. Buying Inverse (Short) ETFs (Recommended)

Instead of borrowing shares to short sell, the bot executes a standard `buy` order on an inverse ETF. This functions like purchasing any ordinary stock, meaning it works on both cash and margin accounts without borrowing fees or locate requirements.

ETF Type	Long Asset (Bullish)	Inverse Equivalent (Bearish)	Leverage Factor
Nasdaq-100	`QQQ`	`PSQ`	$-1\times$ (Unleveraged inverse)
Nasdaq-100 (Leveraged)	`TQQQ` ($+3\times$)	`SQQQ`	$-3\times$ (Triple leveraged inverse)
S&P 500	`SPY`	`SH`	$-1\times$ (Unleveraged inverse)
S&P 500 (Leveraged)	`UPRO` ($+3\times$)	`SPXS`	$-3\times$ (Triple leveraged inverse)

How the Order Logic Looks in LumiBot

```
# When trend is Bullish (Price > 20 SMA):
self.sell_all("PSQ")  # Exit short hedge
self.submit_order(self.create_order("QQQ", bull_qty, "buy"))

# When trend is Bearish (Price < 20 SMA):
self.sell_all("QQQ")  # Exit tech long
self.submit_order(self.create_order("PSQ", bear_qty, "buy"))  # Enter inverse ETF
```

2. Direct Short Selling (Shorting the Long ETF)

If your Alpaca paper account is configured as a **Margin Account**, the bot can execute true short sales directly on `QQQ` or `SPY`.

- **Order Creation:** Calling `self.create_order("QQQ", quantity, "sell")` when your current position is `0` instructs Alpaca to borrow the shares and sell them short.
- **Closing the Short:** Calling `self.create_order("QQQ", quantity, "buy")` submits a buy-to-cover order to return the borrowed shares.

```
# Direct Short Execution
position = self.get_position("QQQ")

# If holding no shares and a sell signal triggers, sell short
if position is None and price < sma:
    short_order = self.create_order("QQQ", quantity, "sell")
    self.submit_order(short_order)
```

```
# If holding a short position and price recovers, buy to cover
elif position is not None and position.quantity < 0 and price > sma:
    cover_order = self.create_order("QQQ", abs(position.quantity), "buy")
    self.submit_order(cover_order)
```

Comparison: Inverse ETFs vs. Direct Short Selling

Feature	Inverse ETFs (`PSQ` / `SQQQ`)	Direct Short Selling (`QQQ` Short)
Order Type	Standard `buy` order	`sell` (to open short)
Account Requirement	Cash or Margin account	Margin account strictly required
Borrowing Fees / Hard-to-Borrow	None	Subject to broker borrow availability and interest
Downside Risk	Capped at 100% of capital deployed	Theoretically unlimited if asset continues to rally
Volatility Drag	High decay over long holding periods	None (tracks pure 1:1 price inverse)

Would you prefer the bot to rotate into an unleveraged inverse ETF like `PSQ` during downtrends, or execute direct short sales on `QQQ`?

User prompt: yes make all the code in one file as liver trader

`Response:`
```
import os
import sys
from dotenv import load_dotenv

# Suppress OpenBLAS thread allocation issues on Windows
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

load_dotenv()

from lumibot.brokers import Alpaca
from lumibot.strategies import Strategy
from lumibot.traders import Trader


# ======================================================================
# STRATEGY 1: Tech Trend (QQQ Long / PSQ Inverse Short Rotation)
# ======================================================================
class QQQRotationLive(Strategy):
    parameters = {
        "bull_symbol": "QQQ",
        "bear_symbol": "PSQ",  # -1x Inverse Nasdaq ETF
        "sma_period": 20,
        "buffer_pct": 0.005,  # 0.5% hysteresis to eliminate false chops
        "allocation": 0.45,  # 45% maximum portfolio cap
    }

    def initialize(self):
```

```
            # Poll every 1 minute during market hours
            self.sleeptime = "1M"

        def on_trading_iteration(self):
            bull = self.parameters["bull_symbol"]
            bear = self.parameters["bear_symbol"]
            sma_period = self.parameters["sma_period"]

            bars = self.get_historical_prices(bull, sma_period, "day")
            bull_price = self.get_last_price(bull)

            if bars is None or len(bars.df) < sma_period or bull_price is None:
                return

            sma = float(bars.df["close"].mean())
            bull_pos = self.get_position(bull)
            bear_pos = self.get_position(bear)

            # Bull Regime: Price breaks above 20 SMA
            if bull_price > sma:
                if bear_pos is not None:
                    self.sell_all(bear)
                    self.log_message(f"[QQQ-Strat] Exited Short ETF {bear}")

                if bull_pos is None:
                    budget = self.get_portfolio_value() * self.parameters["allocation"]
                    qty = int(budget // bull_price)
                    if qty > 0:
                        self.submit_order(self.create_order(bull, qty, "buy"))
                        self.log_message(
                            f"[QQQ-Strat] ENTER LONG -> Buy {qty} {bull} @ ${bull_price:.2f} | 20 SMA: ${sma:.2f}"
                        )

            # Bear Regime: Price drops 0.5% below 20 SMA -> Rotate to Inverse ETF
            elif bull_price < (sma * (1 - self.parameters["buffer_pct"])):
                if bull_pos is not None:
                    self.sell_all(bull)
                    self.log_message(f"[QQQ-Strat] Exited Long ETF {bull}")

                if bear_pos is None:
                    bear_price = self.get_last_price(bear)
                    if bear_price and bear_price > 0:
                        budget = self.get_portfolio_value() * self.parameters["allocation"]
                        qty = int(budget // bear_price)
                        if qty > 0:
                            self.submit_order(self.create_order(bear, qty, "buy"))
                            self.log_message(
                                f"[QQQ-Strat] ENTER SHORT HEDGE -> Buy {qty} {bear} @ ${bear_price:.2f}"
                            )


# ======================================================================
# STRATEGY 2: Broad Market Momentum (SPY Long / SH Inverse Short)
# ======================================================================
class SPYMomentumLive(Strategy):
```

```
parameters = {
    "bull_symbol": "SPY",
    "bear_symbol": "SH",  # -1x Inverse S&P 500 ETF
    "lookback_days": 10,
    "entry_threshold": 0.02,  # +2.0% ROC triggers Long
    "exit_threshold": 0.0,  # ROC below 0% rotates out of Long
    "short_threshold": -0.015,  # -1.5% ROC triggers Inverse Short
    "allocation": 0.45,  # 45% maximum portfolio cap
}

def initialize(self):
    # Poll every 5 minutes during market hours
    self.sleeptime = "5M"

def on_trading_iteration(self):
    bull = self.parameters["bull_symbol"]
    bear = self.parameters["bear_symbol"]
    lookback = self.parameters["lookback_days"]

    bars = self.get_historical_prices(bull, lookback + 1, "day")
    bull_price = self.get_last_price(bull)

    if bars is None or len(bars.df) < (lookback + 1) or bull_price is None:
        return

    past_price = float(bars.df["close"].iloc[0])
    momentum = (bull_price - past_price) / past_price

    bull_pos = self.get_position(bull)
    bear_pos = self.get_position(bear)

    # Bull Regime: 10-day Rate of Change exceeds +2%
    if momentum > self.parameters["entry_threshold"]:
        if bear_pos is not None:
            self.sell_all(bear)
            self.log_message(f"[SPY-Strat] Exited Short ETF {bear}")

        if bull_pos is None:
            budget = self.get_portfolio_value() * self.parameters["allocation"]
            qty = int(budget // bull_price)
            if qty > 0:
                self.submit_order(self.create_order(bull, qty, "buy"))
                self.log_message(
                    f"[SPY-Strat] ENTER LONG -> Buy {qty} {bull} | 10D ROC: {momentum*100:.2f}%"
                )

    # Bear Regime: 10-day Rate of Change deteriorates below -1.5%
    elif momentum < self.parameters["short_threshold"]:
        if bull_pos is not None:
            self.sell_all(bull)
            self.log_message(f"[SPY-Strat] Exited Long ETF {bull}")

        if bear_pos is None:
            bear_price = self.get_last_price(bear)
            if bear_price and bear_price > 0:
```

```
                    budget = self.get_portfolio_value() * self.parameters["allocation"]
                    qty = int(budget // bear_price)
                    if qty > 0:
                        self.submit_order(self.create_order(bear, qty, "buy"))
                        self.log_message(
                            f"[SPY-Strat] ENTER SHORT HEDGE -> Buy {qty} {bear} | 10D ROC: {momentum*100:.2f}%"
                        )

        # Neutral / Chop Regime: Momentum drops below 0% while long
        elif bull_pos is not None and momentum < self.parameters["exit_threshold"]:
            self.sell_all(bull)
            self.log_message(f"[SPY-Strat] ROTATE TO CASH -> Closed {bull} | Momentum flattened")


# ======================================================================
# UNIFIED MULTI-STRATEGY TRADER ORCHESTRATOR
# ======================================================================
if __name__ == "__main__":
    api_key = os.getenv("ALPACA_API_KEY")
    api_secret = os.getenv("ALPACA_API_SECRET")
    is_paper = os.getenv("ALPACA_IS_PAPER", "true").lower() == "true"

    if not api_key or not api_secret:
        print("Error: Missing ALPACA_API_KEY or ALPACA_API_SECRET in .env file.")
        sys.exit(1)

    alpaca_config = {
        "API_KEY": api_key,
        "API_SECRET": api_secret,
        "PAPER": is_paper,
    }

    broker = Alpaca(alpaca_config)

    # Initialize independent strategy workers
    tech_strategy = QQQRotationLive(broker=broker)
    spy_strategy = SPYMomentumLive(broker=broker)

    # Attach both to the unified execution orchestrator
    trader = Trader()
    trader.add_strategy(tech_strategy)
    trader.add_strategy(spy_strategy)

    print("=" * 65)
    print(" UNIFIED MULTI-STRATEGY TRADER (LIVE PAPER MODE)")
    print(" Active Strategy 1: Tech Trend (QQQ Long / PSQ Short) [45%]")
    print(" Active Strategy 2: Market Momentum (SPY Long / SH Short) [45%]")
    print(" Unallocated Buffer: 10% Cash Reserve")
    print("=" * 65)

    trader.run_all()
```

How This Script Executes Trades

Component	Tech Trend Strategy	Broad Market Momentum
Long Asset	`QQQ` (Invesco QQQ Trust)	`SPY` (SPDR S&P 500 ETF)
Short Asset	`PSQ` (ProShares Short QQQ, $-1\times$)	`SH` (ProShares Short S&P 500, $-1\times$)
Allocation Limit	45% max portfolio value	45% max portfolio value
Long Entry Signal	Daily close crosses above 20-day SMA	10-day Rate of Change exceeds **+2.0%**
Short Entry Signal	Daily close drops **0.5%** below 20-day SMA	10-day Rate of Change drops below **-1.5%**
Neutral Rule	Immediate switch (always long `QQQ` or `PSQ`)	Rotates into pure cash when ROC is between 0.0% and -1.5%
Check Frequency	1 Minute (`sleeptime = "1M"`)	5 Minutes (`sleeptime = "5M"`)
Exit Isolation	`self.sell_all(symbol)` (QQQ / PSQ only)	`self.sell_all(symbol)` (SPY / SH only)

Save this file as `live_trader.py` and start it with:

```
python live_trader.py
```

---

*User prompt: (venv) PS D:\alpaca\my_algo_bot> python live_trader.py 2026-09-12 21:10:47,316 | INFO | LumiBot v4.5.91 starting 2026-09-12 21:10:47,402 | INFO | .env file loaded from: D:\alpaca\my_algo_bot\.env 2026-09-12 21:10:49,905 | INFO | [unknown] Waiting for the socket stream connection to be established, method _stream_established must be called 2026-09-12 21:10:50,020 | INFO | [unknown] Waiting for the socket stream connection to be established, method _stream_established must be called 2026-09-12 21:10:52,013 | INFO | [unknown] LUMIBOT_TELEMETRY {"broker":"alpaca","broker_snapshot": {"orders_canceled":0,"orders_error":0,"orders_filled":0,"orders_new":0,"orders_partial":0,"orders_placeholder":0,"orders_unprocessed":0,"positions_tracked":0,"trade_events_rows":0},"strategy_name":"
2026-09-12 21:10:52,124 | INFO | [unknown] LUMIBOT_TELEMETRY {"broker":"alpaca","broker_snapshot": {"orders_canceled":0,"orders_error":0,"orders_filled":0,"orders_new":0,"orders_partial":0,"orders_placeholder":0,"orders_unprocessed":0,"positions_tracked":0,"trade_events_rows":0},"strategy_name":"
== UNIFIED MULTI-STRATEGY TRADER (LIVE PAPER MODE) Active Strategy 1: Tech Trend (QQQ Long / PSQ Short) [45%] Active Strategy 2: Market Momentum (SPY Long / SH Short) [45%] Unallocated Buffer: 10% Cash Reserve
== Traceback (most recent call last): File "D:\alpaca\my_algo_bot\live_trader.py", line 190, in <module> trader.run_all() File "D:\alpaca\my_algo_bot\lumibot\traders\trader.py", line 169, in run_all raise NotImplementedError(NotImplementedError: Running multiple live strategies is not implemented yet. You passed in 2 strategies. (venv) PS D:\alpaca\my_algo_bot>*

Response: LumiBot's `Trader.run_all()` raises a `NotImplementedError` because the framework currently allows only **one** strategy class to be registered with the live trader orchestrator at a time.

To run both systems concurrently in live paper trading, merge both execution routines into a single **Composite Strategy** class. This allows a single strategy instance to manage both the QQQ/PSQ trend rotation and the SPY/SH momentum logic within each iteration loop.

Replace the contents of `live_trader.py` with the unified composite code below:

```
import os
import sys
from dotenv import load_dotenv

# Suppress OpenBLAS thread allocation issues on Windows
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

load_dotenv()

from lumibot.brokers import Alpaca
```

```
from lumibot.strategies import Strategy
from lumibot.traders import Trader


class UnifiedMultiTraderLive(Strategy):
    parameters = {
        # --- Strategy 1: Tech Trend (QQQ / PSQ) ---
        "qqq_bull": "QQQ",
        "qqq_bear": "PSQ",
        "qqq_sma_period": 20,
        "qqq_buffer_pct": 0.005,
        "qqq_allocation": 0.45,
        # --- Strategy 2: Market Momentum (SPY / SH) ---
        "spy_bull": "SPY",
        "spy_bear": "SH",
        "spy_lookback_days": 10,
        "spy_entry_thresh": 0.02,
        "spy_exit_thresh": 0.0,
        "spy_short_thresh": -0.015,
        "spy_allocation": 0.45,
    }

    def initialize(self):
        # Poll market data every 1 minute during trading hours
        self.sleeptime = "1M"
        self.iteration_count = 0

    def on_trading_iteration(self):
        self.iteration_count += 1
        portfolio_value = self.get_portfolio_value()

        # Execute Routine 1: QQQ / PSQ Trend Rotation (runs every minute)
        self._execute_qqq_rotation(portfolio_value)

        # Execute Routine 2: SPY / SH Momentum (runs every 5 minutes to reduce churn)
        if self.iteration_count % 5 == 0:
            self._execute_spy_momentum(portfolio_value)

    # ------------------------------------------------------------------
    # Routine 1: QQQ Breakout & Inverse Rotation
    # ------------------------------------------------------------------
    def _execute_qqq_rotation(self, portfolio_value: float):
        bull = self.parameters["qqq_bull"]
        bear = self.parameters["qqq_bear"]
        sma_period = self.parameters["qqq_sma_period"]

        bars = self.get_historical_prices(bull, sma_period, "day")
        bull_price = self.get_last_price(bull)

        if bars is None or len(bars.df) < sma_period or bull_price is None:
            return

        sma = float(bars.df["close"].mean())
        bull_pos = self.get_position(bull)
        bear_pos = self.get_position(bear)
```

```
        # Bull Regime: Price > 20-day SMA
        if bull_price > sma:
            if bear_pos is not None:
                self.sell_all(bear)
                self.log_message(f"[QQQ-Trend] Exited Short ETF {bear}")

            if bull_pos is None:
                budget = portfolio_value * self.parameters["qqq_allocation"]
                qty = int(budget // bull_price)
                if qty > 0:
                    self.submit_order(self.create_order(bull, qty, "buy"))
                    self.log_message(
                        f"[QQQ-Trend] BUY BULL {qty} {bull} @ ${bull_price:.2f} | 20 SMA: ${sma:.2f}"
                    )

        # Bear Regime: Price drops 0.5% below 20-day SMA
        elif bull_price < (sma * (1 - self.parameters["qqq_buffer_pct"])):
            if bull_pos is not None:
                self.sell_all(bull)
                self.log_message(f"[QQQ-Trend] Exited Long ETF {bull}")

            if bear_pos is None:
                bear_price = self.get_last_price(bear)
                if bear_price and bear_price > 0:
                    budget = portfolio_value * self.parameters["qqq_allocation"]
                    qty = int(budget // bear_price)
                    if qty > 0:
                        self.submit_order(self.create_order(bear, qty, "buy"))
                        self.log_message(
                            f"[QQQ-Trend] BUY BEAR HEDGE {qty} {bear} @ ${bear_price:.2f}"
                        )

    # ------------------------------------------------------------------
    # Routine 2: SPY Momentum & Inverse Rotation
    # ------------------------------------------------------------------
    def _execute_spy_momentum(self, portfolio_value: float):
        bull = self.parameters["spy_bull"]
        bear = self.parameters["spy_bear"]
        lookback = self.parameters["spy_lookback_days"]

        bars = self.get_historical_prices(bull, lookback + 1, "day")
        bull_price = self.get_last_price(bull)

        if bars is None or len(bars.df) < (lookback + 1) or bull_price is None:
            return

        past_price = float(bars.df["close"].iloc[0])
        momentum = (bull_price - past_price) / past_price

        bull_pos = self.get_position(bull)
        bear_pos = self.get_position(bear)

        # Bull Momentum: 10-day ROC > +2.0%
        if momentum > self.parameters["spy_entry_thresh"]:
```

```
            if bear_pos is not None:
                self.sell_all(bear)
                self.log_message(f"[SPY-Momentum] Exited Short ETF {bear}")

            if bull_pos is None:
                budget = portfolio_value * self.parameters["spy_allocation"]
                qty = int(budget // bull_price)
                if qty > 0:
                    self.submit_order(self.create_order(bull, qty, "buy"))
                    self.log_message(
                        f"[SPY-Momentum] BUY BULL {qty} {bull} | 10D ROC: {momentum*100:.2f}%"
                    )

        # Bear Breakdown: 10-day ROC < -1.5%
        elif momentum < self.parameters["spy_short_thresh"]:
            if bull_pos is not None:
                self.sell_all(bull)
                self.log_message(f"[SPY-Momentum] Exited Long ETF {bull}")

            if bear_pos is None:
                bear_price = self.get_last_price(bear)
                if bear_price and bear_price > 0:
                    budget = portfolio_value * self.parameters["spy_allocation"]
                    qty = int(budget // bear_price)
                    if qty > 0:
                        self.submit_order(self.create_order(bear, qty, "buy"))
                        self.log_message(
                            f"[SPY-Momentum] BUY BEAR HEDGE {qty} {bear} @ ${bear_price:.2f}"
                        )

        # Neutral / Mean-Reverting: Momentum drops below 0%
        elif bull_pos is not None and momentum < self.parameters["spy_exit_thresh"]:
            self.sell_all(bull)
            self.log_message(f"[SPY-Momentum] ROTATE TO CASH -> Closed {bull}")


# ============================================================================
# RUNNER
# ============================================================================
if __name__ == "__main__":
    api_key = os.getenv("ALPACA_API_KEY")
    api_secret = os.getenv("ALPACA_API_SECRET")
    is_paper = os.getenv("ALPACA_IS_PAPER", "true").lower() == "true"

    if not api_key or not api_secret:
        print("Error: Missing ALPACA_API_KEY or ALPACA_API_SECRET in .env file.")
        sys.exit(1)

    alpaca_config = {
        "API_KEY": api_key,
        "API_SECRET": api_secret,
        "PAPER": is_paper,
    }

    broker = Alpaca(alpaca_config)
```

```
# Register only ONE composite strategy instance
strategy = UnifiedMultiTraderLive(broker=broker)

trader = Trader()
trader.add_strategy(strategy)

print("=" * 65)
print(" UNIFIED MULTI-STRATEGY TRADER (COMPOSITE LIVE PAPER MODE)")
print(" Active: [QQQ/PSQ Trend Rotation] + [SPY/SH Momentum Rotation]")
print(" Allocation: 45% Tech / 45% Market / 10% Cash Reserve")
print("=" * 65)

trader.run_all()
```

Key Architectural Fixes

- **Single Strategy Registration:** By subclassing `UnifiedMultiTraderLive` once, only one strategy object is passed to `trader.add_strategy(strategy)`. This bypasses LumiBot's `NotImplementedError` while running both trading models concurrently.
- **Internal Loop Staggering:** The main execution loop runs every 60 seconds (`self.sleeptime = "1M"`). The QQQ breakout logic evaluates on every tick, while the SPY momentum logic runs on modulo check (`self.iteration_count % 5 == 0`), preventing redundant API calls and rate-limit throttling.
- **Dedicated Symbol Liquidation:** Calls to `self.sell_all(bear)` or `self.sell_all(bull)` only sell the specific ticker associated with that sub-routine, preventing trades in QQQ from interfering with positions in SPY.

Run the updated file in your terminal:

```
python live_trader.py
```

Would you like to start this script now to confirm that the single composite strategy connects and enters sleep mode without raising the error?

User prompt: (venv) PS D:\alpaca\my_algo_bot> python live_trader.py 2026-09-12 21:13:02,220 | INFO | LumiBot v4.5.91 starting 2026-09-12 21:13:02,262 | INFO | .env file loaded from: D:\alpaca\my_algo_bot\.env 2026-09-12 21:13:04,771 | INFO | [unknown] Waiting for the socket stream connection to be established, method _stream_established must be called 2026-09-12 21:13:04,884 | INFO | [unknown] Waiting for the socket stream connection to be established, method _stream_established must be called 2026-09-12 21:13:06,882 | INFO | [unknown] LUMIBOT_TELEMETRY {"broker":"alpaca","broker_snapshot": {"orders_canceled":0,"orders_error":0,"orders_filled":0,"orders_new":0,"orders_partial":0,"orders_placeholder":0,"orders_unprocessed":0,"positions_tracked":0,"trade_events_rows":0},"strategy_name":ı 2026-09-12 21:13:06,989 | INFO | [unknown] LUMIBOT_TELEMETRY {"broker":"alpaca","broker_snapshot": {"orders_canceled":0,"orders_error":0,"orders_filled":0,"orders_new":0,"orders_partial":0,"orders_placeholder":0,"orders_unprocessed":0,"positions_tracked":0,"trade_events_rows":0},"strategy_name":ı == UNIFIED MULTI-STRATEGY TRADER (COMPOSITE LIVE PAPER MODE) Active: [QQQ/PSQ Trend Rotation] + [SPY/SH Momentum Rotation] Allocation: 45% Tech / 45% Market / 10% Cash Reserve == 2026-09-12 21:13:07,938 | INFO | [UnifiedMultiTraderLive] Strategy UnifiedMultiTraderLive is initializing 2026-09-12 21:13:08,848 | WARNING | lumibot_logger.py:warning:613 | [UnifiedMultiTraderLive] LUMIWEALTH_API_KEY not set. Not sending an update to the cloud because lumiwealth_api_key is not set. If you would like to be able to track your bot performance on www.botspot.trade, please set the lumiwealth_api_key parameter in the strategy initialization or the LUMIWEALTH_API_KEY environment variable. 2026-09-12 21:13:09,371 | INFO | [UnifiedMultiTraderLive] Sleeping until the market opens

Response: The composite runner initialized cleanly, bypassed the `NotImplementedError`, and is now safely hibernating in standby mode until the US market opening bell.

What Will Happen on Monday

The bot will wake up automatically on **Monday at 9:30 AM EDT (7:00 PM IST)** and execute the following routines:

Execution Cycle	Routine	Logic Evaluated	Action Taken
Every 1 Minute	**Tech Trend Rotation**	• QQQ > 20 SMA • QQQ < (20 SMA × 0.995)	• Sells PSQ, enters **QQQ** (45% cap) • Sells QQQ, rotates to **PSQ** (45% cap)
Every 5 Minutes	**Market Momentum**	• SPY 10D ROC > +2.0% • SPY 10D ROC < -1.5% • SPY 10D ROC < 0.0%	• Sells SH, enters **SPY** (45% cap) • Sells SPY, rotates to **SH** (45% cap) • Rotates SPY allocation to Cash
Continuous	**Cash Protection**	Minimum 10% cash buffer preserved	Prevents buying power rejections and margin calls

All order executions, fills, and portfolio rebalances will reflect live on your Alpaca Paper Dashboard.

How to Manage the Process Over the Weekend

- **Leave It Running:** If your PC remains powered on and connected to the internet, you can minimize the terminal. The process will remain idle and begin evaluating bars automatically at 7:00 PM IST Monday.
- **Stop and Relaunch Later:** You can safely press `Ctrl + C` to stop the process now. To resume before market open, run:

  ```
  python live_trader.py
  ```

Would you like to integrate automated trade notifications (via Telegram or Discord webhooks) into the script so you receive alerts on your phone whenever an order fills?
